Asymmetric Auto-Encoder Bottlenecks for Self-Supervised Latent Representation Learning

Course Code - BCSE332L

Course Title - Deep Learning

Instructor - Prof. VARALAKSHMI M

Team Members

Kashvi Agrawal 23BDS0336

Sudeep Sharma 23BDS0315

GitHub repository: https://github.com/kashviagrawal04/ASYMMETRIC-ENCODER

Live demo: https://kashviagrawal04.github.io/ASYMMETRIC-ENCODER/demo/

Abstract

Auto-encoders learn compact codes without labels, and recent self-supervised methods often give the decoder less capacity than the encoder. This project asks how the capacity asymmetry between encoder and decoder shapes the code that is learned. We built an auto-encoder framework from scratch in NumPy with independently configurable encoder and decoder depth, and judged frozen latents on the 8×8 Digits dataset with a linear probe, low-label probes, clustering, neighbourhood preservation, effective rank, robustness and computational cost, over 5 seeds with a fresh data split per seed and paired statistics. A symmetric deep model was best at narrow bottlenecks (probe accuracy 0.740 against 0.545 for a deep encoder with a single-layer decoder at 2 dimensions), and the light-decoder model suffered dimensional collapse (effective rank 1.01 of 2). A random-encoder control shows that training matters most when the bottleneck is narrow. A latent regulariser removes the collapse and repairs clustering but not linear separability, whereas one hidden decoder layer fixes both. From 16 dimensions upwards all models, PCA and raw pixels are indistinguishable on this easy dataset.

Keywords: auto-encoder, *asymmetric architecture*, *latent representation*, *bottleneck*, *dimensional collapse*, *linear probe*, *effective rank*, *NumPy*.

Table of Contents

Abstract

Summary of Findings

1. Problem Statement

2. Work Done

3. Implementation (3.1 Data, 3.2 Models, 3.3 Training, regulariser and evaluation)

4. Results (4.1 Main sweep to 4.10 Random-encoder control)

5. Discussion (5.1 Hypotheses to 5.5 Next steps)

Reproducibility

References

Summary of Findings

Auto-encoders learn representations without labels by squeezing an input through a bottleneck and reconstructing it [1]. Most designs mirror the encoder and decoder; recent self-supervised work such as masked auto-encoders [3] often breaks that symmetry, frequently with a light decoder, on the argument that a weak decoder forces the encoder to do the semantic work. This project tests that idea in a controlled, fully reproducible setting. I built an auto-encoder framework from scratch in NumPy in which encoder and decoder depth are set independently, and evaluated it on frozen latents with a linear probe, low-label probes, k-NN, label-free clustering, neighbourhood preservation, latent effective rank and robustness to shifts, rotations and noise, with every comparison paired by seed.

Headline results

- **A light decoder did not help.** Symmetric deep auto-encoders were best whenever the bottleneck was narrow: probe accuracy 0.740 / 0.852 / 0.935 at k = 2 / 4 / 8, versus 0.545 / 0.817 / 0.912 for a deep encoder with a single-layer decoder.
- **Deep encoder + single-layer decoder suffers dimensional collapse.** Effective rank 1.01 of 2 at k = 2 and 1.23 of 4 at k = 4, and clustering quality (NMI) falls from 0.65 to 0.36 at k = 2.
- **Diagnosis.** The collapse is an early training plateau (rank drops to about 1 within the first 25 to 100 epochs) that longer training or a higher learning rate only partly escapes. A variance-covariance regulariser removes the collapse and repairs clustering, but does not restore linear separability. Giving the decoder one hidden layer does both.
- **Limits of the evidence.** On this small, easy dataset (raw pixels already give 0.969 probe accuracy) every model, PCA included, is indistinguishable at k ≥ 16, so the conclusions concern narrow bottlenecks only.

1. Problem Statement

Given unlabeled data $x \in \mathbb{R}^{64}$, learn an encoder $f: \mathbb{R}^{64} \to \mathbb{R}^{k}$ and decoder $g: \mathbb{R}^{k} \to \mathbb{R}^{64}$ by minimising reconstruction error $\|x - g(f(x))\|^2$. The objective is not reconstruction itself but a code $z = f(x)$ that is useful downstream without label supervision.

Research question. At a fixed bottleneck width k, how does the capacity asymmetry between encoder and decoder affect the quality and geometry of the learned latent representation, and why?

Hypotheses

- **H1 (light decoder helps).** A weak decoder cannot do much work, so the encoder must produce a more informative, linearly separable code.
- **H2 (heavy decoder compensates).** A strong decoder can reconstruct from a simple encoder's code, giving good reconstruction but a less structured latent space.
- **H3 (noise helps).** A masking-noise denoising objective regularises the latent space, most where the decoder is weak.
- **H4 (collapse is the mechanism).** If a light decoder hurts, it does so by letting the latent space collapse to fewer effective dimensions; remedies that restore effective rank should restore accuracy.

2. Work Done

Version 1 of this project compared four architectures on one metric family and left the main finding unexplained. Version 2 extends it in five directions:

- **Broader architecture space.** A full 3 × 3 encoder-depth × decoder-depth grid (9 architectures, 3 bottleneck widths) instead of four hand-picked models.
- **Mechanism, not just ranking.** A dedicated diagnosis of the light-decoder collapse: per-epoch effective-rank traces, longer training, a higher learning rate, a one-layer decoder, and a new variance-covariance latent regulariser with strength chosen on a validation split.
- **Richer evaluation.** Added 1 / 5 / 10 / 25 %-label probes, K-means NMI, trustworthiness, and robustness to 1-px shifts, 15° rotations and Gaussian noise.
- **More honest statistics.** Every seed now draws its own stratified train/test split (so variance includes data-split variance) and models are compared with paired differences, bootstrap confidence intervals and paired t-tests.
- **Engineering.** A package layout with 18 automated tests (finite-difference gradient checks for every architecture and the regulariser), CI, a Makefile, a license, and a self-contained interactive latent-space explorer whose decoder runs in JavaScript.

3. Implementation

3.1 Data

scikit-learn Digits [12, 14]: 1,797 grayscale 8×8 images (64 features, 10 classes), scaled to [0, 1], split 75/25 (1,347 train / 450 test) with class stratification. The split depends on the seed. Three transformed copies of each test set probe invariance: a 1-pixel right shift, a 15° rotation and additive Gaussian noise ($\sigma = 0.15$). MNIST was the original plan, but dataset downloads were blocked in my environment, so Digits (bundled with scikit-learn) was used throughout.

3.2 Models

A from-scratch NumPy [13] library (dense layers, manual backpropagation [4], Adam [6]). Encoder and decoder depth are independent: depth 0 is a single linear/sigmoid layer, depth 1 adds a 128-unit ReLU layer, depth 2 adds 128-64 (encoder) or 64-128 (decoder). The bottleneck is linear with width $k \in \{2, 4, 8, 16, 32\}$ and the output is a sigmoid. Named variants: symmetric (2/2), heavy encoder / light decoder (2/0), light encoder / heavy decoder (0/2) and a fully linear AE. Parameter counts at k = 2:

Architecture (enc/dec depth)	Encoder params	Decoder params
e0d0 (0/0)	130	192
e0d1 (0/1)	130	8,640
e0d2 (0/2)	130	16,768
e1d0 (1/0)	8,578	192
e1d1 (1/1)	8,578	8,640
e1d2 (1/2)	8,578	16,768
e2d0 (2/0)	16,706	192
e2d1 (2/1)	16,706	8,640
e2d2 (2/2)	16,706	16,768

3.3 Training, regulariser and evaluation

Mean-squared-error loss, Adam [6] (learning rate 1e-3), batch size 64, 300 epochs, He initialisation [11]; identical hyper-parameters for every architecture. The denoising ablation [2] zeroes each input pixel with probability 0.3 and reconstructs the clean image. The latent regulariser is VICReg-style [5]: $L_{reg} = \text{mean}_j \max(0, 1 - \text{std}_j)^2 + (1/k) \Sigma_{a \neq b} C_{ab}^2$, where C is the batch covariance of the latents; it is added to the loss with weight λ and its gradient is derived by hand and verified against finite differences. λ was selected from {0.01, 0.1, 1.0} on a validation split carved out of the training data (λ = 0.01); the test set was never used for selection.

Metrics (labels are used only here, never in training):

- **Linear probe** (standardise + logistic regression on frozen latents), also with 1, 5, 10 and 25 % of training labels [9].
- **5-NN accuracy and K-means NMI.** NMI clusters the latents into 10 groups without labels and compares them with the true classes.
- **Trustworthiness.** With 10 neighbours: how well local neighbourhoods of the input are preserved in the latent space [8].
- **Effective rank** $(\Sigma\lambda)^2 / \Sigma\lambda^2$ of the latent covariance eigenvalues: equal to k for an isotropic code, close to 1 for a code collapsed onto a line [7].
- **Robustness** (probe trained on clean latents, scored on shifted / rotated / noisy test latents) and reconstruction MSE.

Baselines are PCA at every k and the raw 64 pixels. Experiments run in five stages (main sweep, denoising ablation, depth grid, collapse diagnosis, regularised sweep): 5 seeds for the main, denoising and regularised stages, 3 seeds for the depth grid and the diagnosis.

4. Results

4.1 Main sweep: linear-probe accuracy

k	Symmetric	Heavy enc. / light dec.	Light enc. / heavy dec.	Linear AE	PCA
2	**0.740 ± 0.061**	0.545 ± 0.044	0.640 ± 0.050	0.554 ± 0.055	0.596 ± 0.023
4	**0.852 ± 0.049**	0.817 ± 0.024	0.827 ± 0.037	0.803 ± 0.020	0.811 ± 0.015
8	**0.935 ± 0.006**	0.912 ± 0.018	0.909 ± 0.012	0.903 ± 0.008	0.903 ± 0.013
16	0.950 ± 0.010	**0.958 ± 0.012**	0.952 ± 0.006	0.954 ± 0.010	0.952 ± 0.012
32	0.971 ± 0.006	**0.972 ± 0.004**	0.968 ± 0.006	0.968 ± 0.005	0.966 ± 0.006

Test accuracy of a linear probe on frozen latents, mean ± sd over 5 seeds (each seed has its own split). Raw pixels: 0.969. Best per row in bold.

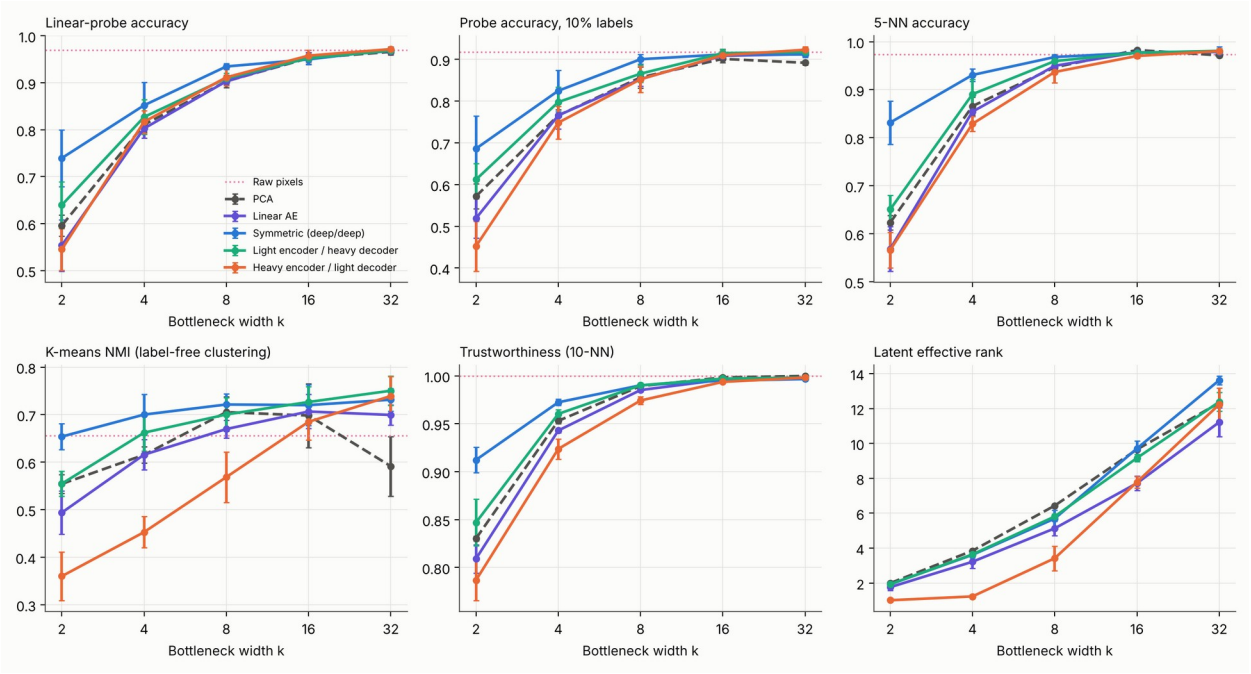


Figure 1. Six representation-quality metrics versus bottleneck width (mean ± sd, 5 seeds). Dotted line: raw pixels.

4.2 Is the difference real? Paired comparison with the symmetric model

Comparison	k=2	k=4	k=8	k=16
Heavy enc. / light dec. − Symm., probe acc.	-0.194 (p=0.000)*	-0.035 (p=0.142)	-0.023 (p=0.012)*	+0.008 (p=0.006)*
Light enc. / heavy dec. − Symm., probe acc.	-0.100 (p=0.065)	-0.025 (p=0.490)	-0.025 (p=0.001)*	+0.003 (p=0.529)
PCA − Symm., probe acc.	-0.144 (p=0.004)*	-0.041 (p=0.175)	-0.032 (p=0.001)*	+0.003 (p=0.637)
Heavy enc. / light dec. − Symm., NMI	-0.294 (p=0.000)*	-0.247 (p=0.000)*	-0.153 (p=0.002)*	-0.035 (p=0.199)
Light enc. / heavy dec. − Symm., NMI	-0.099 (p=0.002)*	-0.038 (p=0.053)	-0.021 (p=0.242)	+0.007 (p=0.755)
PCA − Symm., NMI	-0.100 (p=0.002)*	-0.085 (p=0.005)*	-0.016 (p=0.450)	-0.022 (p=0.374)

*Mean paired difference over 5 seeds (same seed = same split) and paired t-test p-value; * = p < 0.05. With n = 5 these tests are weak; the confidence intervals are in results/stats_vs_symmetric.csv.*

4.3 Several metrics at a glance

k = 2	Probe	1% labels	10% labels	5-NN	NMI	Trust.	Eff. rank	Rec. MSE
Symmetric	0.740	0.520	0.686	0.832	0.654	0.912	1.93	0.034
Heavy enc. / light dec.	0.545	0.315	0.452	0.566	0.360	0.787	1.01	0.055
Light enc. / heavy dec.	0.640	0.481	0.612	0.651	0.555	0.847	1.93	0.042
Linear AE	0.554	0.396	0.519	0.568	0.494	0.809	1.76	0.054
PCA	0.596	0.487	0.572	0.623	0.554	0.830	2.00	0.052

k = 8	Probe	1% labels	10% labels	5-NN	NMI	Trust.	Eff. rank	Rec. MSE
Symmetric	0.935	0.673	0.900	0.968	0.721	0.990	5.69	0.012
Heavy enc. / light dec.	0.912	0.580	0.852	0.937	0.569	0.974	3.42	0.021
Light enc. / heavy dec.	0.909	0.649	0.866	0.960	0.700	0.990	5.81	0.013
Linear AE	0.903	0.645	0.853	0.950	0.670	0.985	5.13	0.025
PCA	0.903	0.635	0.857	0.947	0.705	0.989	6.43	0.024

Raw pixels: probe 0.969, 1% labels 0.562, 10% labels 0.918, NMI 0.656.

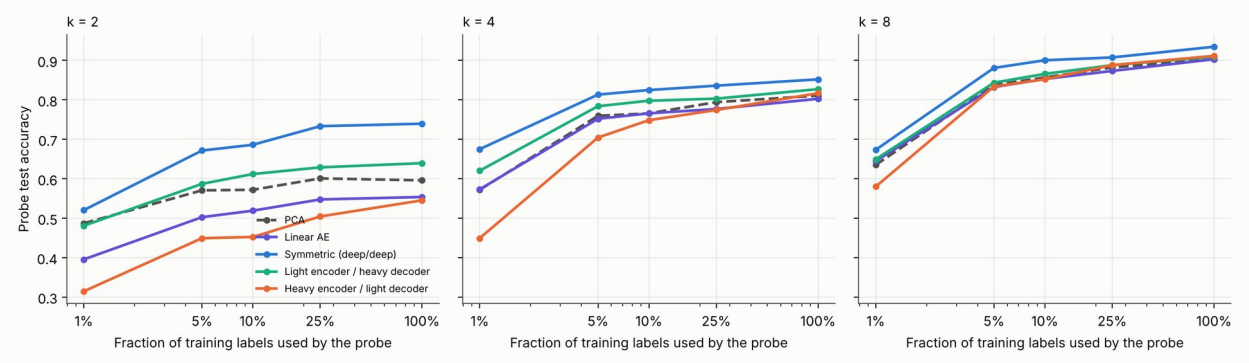


Figure 2. Probe accuracy as a function of how many labels the probe sees.

4.4 Robustness

k = 8	Clean	1-px shift	15° rotation	Noise σ=0.15
Symmetric	0.935	0.420	0.651	0.868
Heavy enc. / light dec.	0.912	0.366	0.714	0.848
Light enc. / heavy dec.	0.909	0.344	0.611	0.842
Linear AE	0.903	0.333	0.681	0.868
PCA	0.903	0.313	0.679	0.882
Raw pixels	0.969	0.392	0.306	0.641

Probe accuracy on transformed test images (probe trained on clean data only).

4.5 Encoder-depth × decoder-depth grid

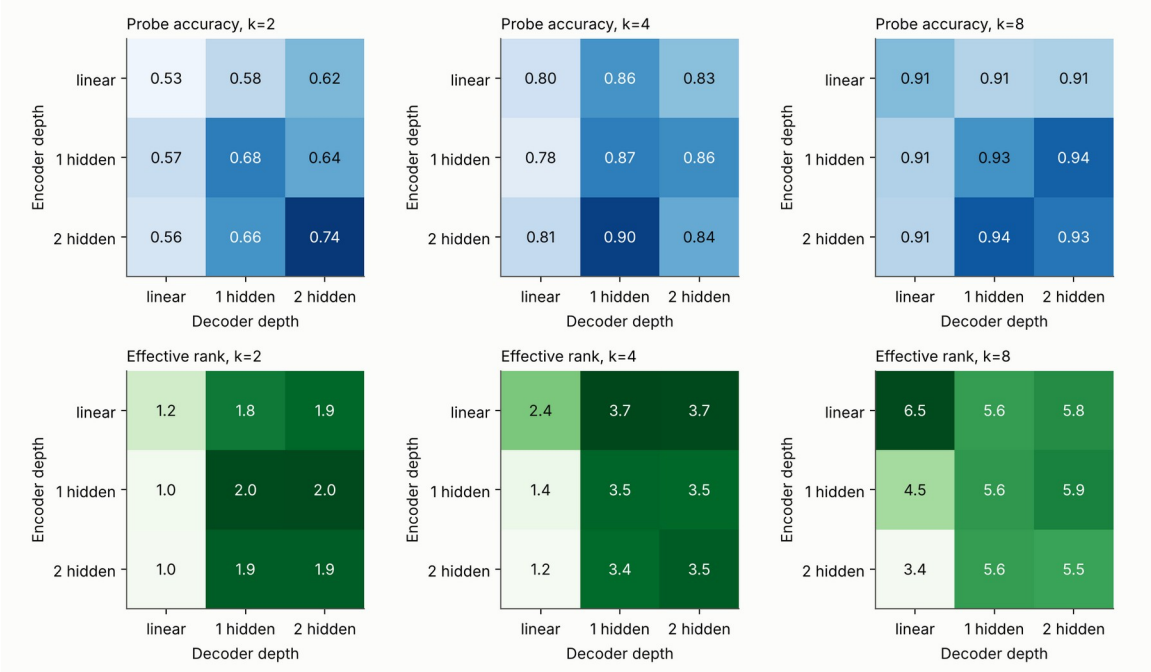


Figure 3. Probe accuracy (top) and effective rank (bottom) for all nine depth combinations (mean of 3 seeds).

Architecture	k = 2	k = 4	k = 8
e0d0 (enc 0 / dec 0)	0.527 ± 0.105	0.797 ± 0.007	0.915 ± 0.009
e0d1 (enc 0 / dec 1)	0.580 ± 0.063	0.856 ± 0.016	0.907 ± 0.018
e0d2 (enc 0 / dec 2)	0.622 ± 0.062	0.828 ± 0.051	0.908 ± 0.014
e1d0 (enc 1 / dec 0)	0.571 ± 0.040	0.784 ± 0.027	0.907 ± 0.011
e1d1 (enc 1 / dec 1)	0.680 ± 0.010	0.867 ± 0.005	0.926 ± 0.014
e1d2 (enc 1 / dec 2)	0.639 ± 0.104	0.860 ± 0.027	0.938 ± 0.008
e2d0 (enc 2 / dec 0)	0.556 ± 0.026	0.805 ± 0.001	0.906 ± 0.020
e2d1 (enc 2 / dec 1)	0.660 ± 0.038	0.901 ± 0.015	0.940 ± 0.008
e2d2 (enc 2 / dec 2)	0.741 ± 0.038	0.841 ± 0.059	0.933 ± 0.008

Probe accuracy, mean ± sd over 3 seeds.

4.6 Diagnosing the collapse

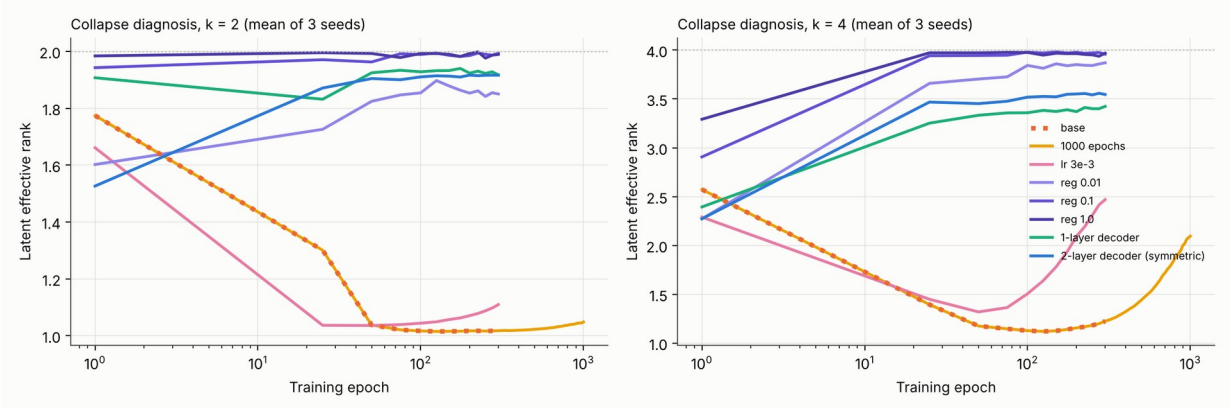


Figure 4. Latent effective rank during training for the heavy-encoder / light-decoder model and its remedies (mean of 3 seeds).

Variant (deep encoder)	k=2 val probe	k=2 test probe	k=2 eff. rank	k=4 val probe	k=4 test probe	k=4 eff. rank
base	0.540	0.556	1.02	0.785	0.805	1.22
1000 epochs	0.579	0.579	1.05	0.809	0.829	2.09
lr 3e-3	0.577	0.601	1.11	0.830	0.836	2.47
reg 0.01	0.584	0.595	1.85	0.780	0.799	3.87
reg 0.1	0.530	0.554	1.99	0.752	0.763	3.96
reg 1.0	0.465	0.456	1.99	0.781	0.777	3.97
1-layer decoder	0.667	0.660	1.92	0.869	0.901	3.42
2-layer decoder (symmetric)	0.689	0.741	1.92	0.869	0.841	3.55

Heavy-encoder remedies, mean of 3 seeds. Validation probe (from a held-out part of the training data) was used to select λ = 0.01; test columns are reported for completeness.

4.7 Latent regulariser across all architectures

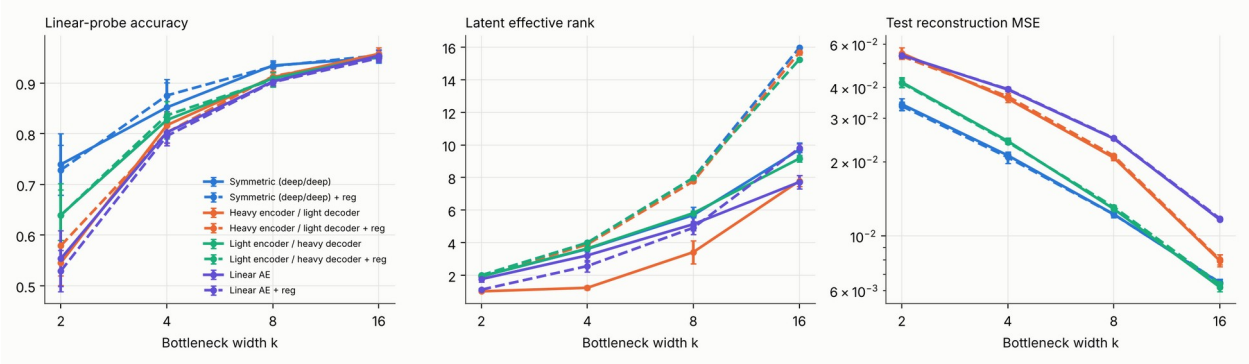


Figure 5. Plain (solid) vs regularised (dashed, λ = 0.01) models, 5 seeds.

Model	k	Probe	NMI	Effective rank
Heavy enc. / light dec.	2	0.545 → 0.579	0.360 → 0.506	1.01 → 1.91
Heavy enc. / light dec.	4	0.817 → 0.803	0.453 → 0.621	1.23 → 3.90
Heavy enc. / light dec.	8	0.912 → 0.913	0.569 → 0.683	3.42 → 7.76
Heavy enc. / light dec.	16	0.958 → 0.953	0.685 → 0.691	7.78 → 15.67
Symmetric	2	0.740 → 0.728	0.654 → 0.641	1.93 → 1.99
Symmetric	4	0.852 → 0.876	0.700 → 0.717	3.61 → 3.99
Symmetric	8	0.935 → 0.933	0.721 → 0.683	5.69 → 7.98
Symmetric	16	0.950 → 0.955	0.720 → 0.688	9.71 → 15.96

Plain → regularised (mean of 5 seeds).

4.8 Reconstruction, latent geometry and denoising

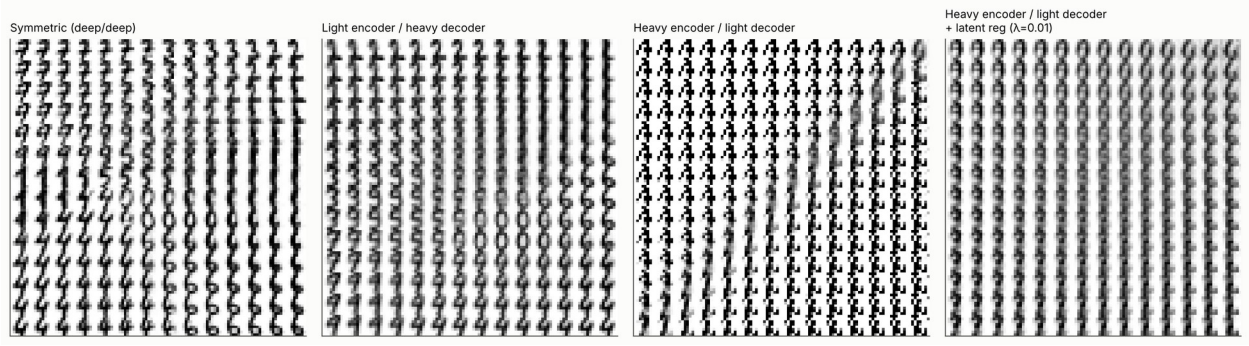


Figure 6. What each decoder paints over the 2-D latent plane (k = 2). The collapsed heavy-encoder model uses a thin line of it.

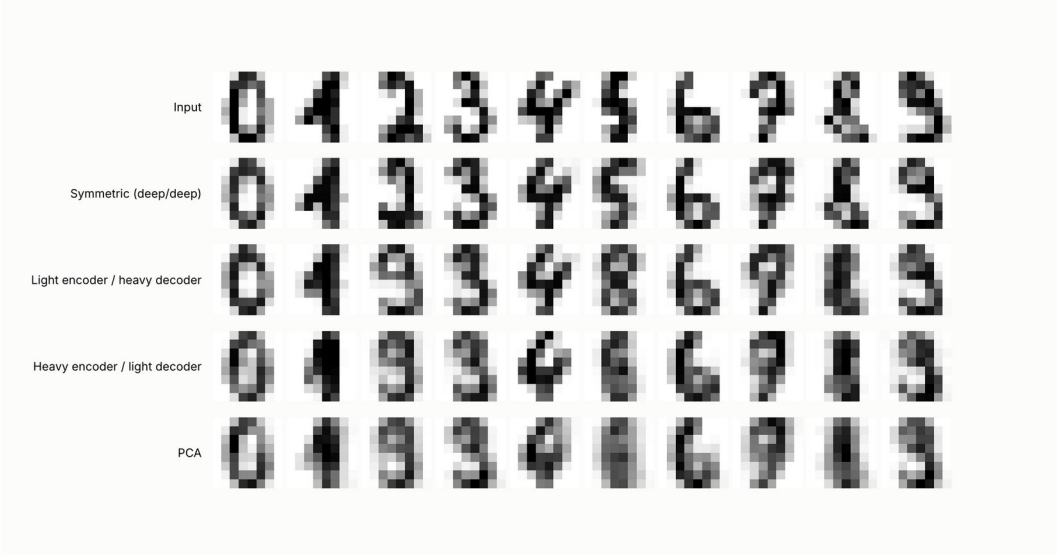


Figure 7. Reconstructions of one test digit per class at k = 4.

Model	k = 8	k = 16
Symmetric	0.935 → 0.938	0.950 → 0.960
Heavy enc. / light dec.	0.912 → 0.924	0.958 → 0.950
Light enc. / heavy dec.	0.909 → 0.904	0.952 → 0.948
Linear AE	0.903 → 0.900	0.954 → 0.950

Probe accuracy, clean training → 30% masking-noise training (5 seeds).

4.9 Computational cost

k = 8	Encoder params	Decoder params	MACs per image	Rec. MSE	Probe acc.
Symmetric	17,096	17,152	33,792	0.0123	0.935
Heavy enc. / light dec.	17,096	576	17,408	0.0208	0.912
Light enc. / heavy dec.	520	17,152	17,408	0.0129	0.909
Linear AE	520	576	1,024	0.0248	0.903
PCA	512	512	1,024	0.0242	0.903

Cost of encoding and decoding one image (multiply-accumulates = number of weights in the fully connected layers) against what it buys.

For fully connected layers the number of multiply-accumulate operations equals the number of weights, so parameter count is a faithful cost proxy here (this is not true for convolutional networks). The symmetric model costs about twice as much as the light-encoder / heavy-decoder model (33,792 vs 17,408 operations) for a similar reconstruction error and 0.026 more probe accuracy. The heavy-encoder / light-decoder model costs the same as the light-encoder / heavy-decoder model but reconstructs much worse (MSE 0.0208 vs 0.0129) and clusters worse (NMI 0.569 vs 0.700), with no reliable probe advantage (0.912 vs 0.909), so it is the less efficient of the two asymmetric designs. The linear models and PCA are about 30 times cheaper than the symmetric model.

4.10 Does training matter? Random-encoder control

k	Random deep encoder	Random linear encoder	Symmetric	Heavy enc.	Heavy dec.	Linear AE
2	0.331	0.328	0.740	0.545	0.640	0.554
4	0.503	0.493	0.852	0.817	0.827	0.803
8	0.683	0.692	0.935	0.912	0.909	0.903
16	0.825	0.854	0.950	0.958	0.952	0.954
32	0.904	0.948	0.971	0.972	0.968	0.968

Linear-probe accuracy of untrained (randomly initialised) encoders against trained models, mean of 5 seeds. Models that share an encoder architecture share the same random encoder.

An untrained encoder is a useful floor: it shows how much of a probe score comes from training rather than from the architecture and the data alone. Training raises probe accuracy at every width. The paired gain of the symmetric model over a random deep encoder is +0.408 at k = 2, +0.349 at k = 4, +0.252 at k = 8 and +0.124 at k = 16, and all gains up to k = 16 are significant (paired $p < 0.01$ for every architecture). At k = 32 the gain shrinks to +0.067 for the deep encoder and +0.020 for the linear one: a random linear projection to 32 dimensions already scores 0.948, close to the 0.969 of raw pixels.

5. Discussion

5.1 Hypotheses, one by one

- **H1 (light decoder helps): not supported.** At k = 2, 4, 8 the deep-encoder / single-layer-decoder model scored 0.545, 0.817, 0.912 against 0.740, 0.852, 0.935 for the symmetric model. The gap is significant at k = 2 (−0.194, $p = 0.0004$) and k = 8 (−0.023, $p = 0.012$); at k = 4 it is of the same size as the seed spread ($p = 0.14$). It is far larger and clearer on clustering: NMI is lower by 0.29, 0.25 and 0.15 at k = 2, 4, 8 (all $p < 0.002$).
- **H2 (heavy decoder compensates): partly supported.** The light-encoder / heavy-decoder model reconstructs about as well as the symmetric one (MSE 0.013 vs 0.012 at k = 8, against 0.025 for the linear AE), so decoder depth, not encoder depth, drives reconstruction quality. Its probe accuracy (0.909 at k = 8) is only slightly above PCA (0.903): good reconstruction from a linear encoder does not yield a better-structured code.
- **H3 (noise helps): not supported.** Masking noise changed probe accuracy by at most 0.012 in either direction (k = 8 and 16, all four architectures), with no consistent sign, which is within seed noise.
- **H4 (collapse is the mechanism): partly supported, and the more interesting result.** See 5.2.

5.2 What the collapse is, and is not

With a single-layer decoder, a deep encoder maps the data onto an almost one-dimensional curve: effective rank 1.01 of 2 at k = 2, 1.23 of 4 at k = 4 and 3.42 of 8 at k = 8 (symmetric: 1.93 / 3.61 / 5.69). This is a form of dimensional collapse, a failure mode studied in contrastive self-supervised learning [10]. The decoded-manifold figure shows the consequence: the digit classes sit along one line and most of the latent plane decodes to meaningless blobs.

- **It is an optimisation plateau, not a hard capacity limit.** The rank trace falls to about 1 within the first 25 to 100 epochs. At k = 4 it recovers slowly, reaching about 2.1 after 1000 epochs, or about 2.5 within 300 epochs at a three times higher learning rate. At k = 2 it had not recovered after 1000 epochs.
- **Fixing the rank is not enough.** The regulariser lifts the effective rank to 3.9 of 4 and 1.9 of 2, yet the validation probe accuracy barely moves (k = 4: 0.780 vs 0.785 for the base model) and can fall with a stronger weight. The code becomes decorrelated, not more informative for classification.
- **Decoder expressiveness is what matters.** One hidden layer in the decoder lifts validation probe accuracy to 0.869 at k = 4 and 0.667 at k = 2, matching the symmetric model at k = 4 (0.869) and closing most of the gap at k = 2. This is the only remedy that restores both rank and accuracy. In the depth grid, every single-layer-decoder architecture is weaker than its one-hidden-layer-decoder counterpart at k = 2 and 4 (for the deepest encoder, 0.556 vs 0.660 at k = 2 and 0.805 vs 0.901 at k = 4), and at k = 4 the 2/1 architecture (0.901) was at least as good as the full 2/2 model (0.841, sd 0.059; that difference is within seed noise).
- **Regularisation does repair clustering.** For the collapsed model the regulariser improves NMI by 0.146 (k = 2, p = 0.003), 0.169 (k = 4, p = 0.0006) and 0.114 (k = 8, p = 0.02) and improves probe accuracy at k = 2 (+0.034, p = 0.03), but not at k ≥ 4. On healthy models it forces near-isotropic codes (effective rank 7.98 of 8) and slightly hurts clustering (symmetric, k = 8: NMI −0.038, p = 0.03), so it is a remedy for collapse rather than a general improvement.

My interpretation, which the experiments support but do not prove, is that a single-layer decoder can only read out a (sigmoid-squashed) linear function of z, so the encoder is pushed towards a nearly linear, low-dimensional solution early in training, and the plateau it then sits on is hard to leave. This differs from masked auto-encoders at scale [3], where the decoder is light relative to a huge encoder but still multi-layer and the task (reconstructing heavily masked inputs) is hard. A one-layer decoder on 8×8 digits is probably too weak, not just lighter, so these results say little about that regime.

5.3 Other observations

- **Depth on both sides matters most at narrow bottlenecks.** At k = 2 the symmetric model leads the grid (0.741; the linear/linear model is at 0.527) and symmetric also leads on 5-NN accuracy (0.83 vs 0.57 for the heavy-encoder model) and trustworthiness (0.91 vs 0.79). By k = 8 any architecture with a non-linear layer on both sides is at 0.93 to 0.94, and by k = 16 all models converge.
- **Label efficiency.** With one labelled example per class (1% of labels), probes on the k = 8 latents reach 0.673 (symmetric) and 0.58 to 0.65 (others) versus 0.562 on raw pixels, with 8 times fewer features. With 10% of labels raw pixels (0.918) already catch up with the best latents (0.900), and at 100% of labels pixels win (0.969 vs 0.935 at k = 8).
- **Robustness.** At k = 8 the bottleneck gives a large noise-robustness gain: 0.84 to 0.88 on noisy inputs versus 0.64 for pixels. No representation is shift-invariant: a one-pixel shift drops every model below 0.43, because nothing in training asks for invariance. Rotation robustness varies (heavy-encoder 0.714, symmetric 0.651, pixels 0.306) and was not predicted by clean accuracy.
- **Wide bottlenecks.** At k ≥ 16 all models, PCA and raw pixels reach 0.95 to 0.97. The heavy-encoder model is nominally ahead of the symmetric model at k = 16 (+0.008, p = 0.006), a gap too small to interpret on this dataset.

- **Training matters most where compression is severe.** The random-encoder control (4.10) shows that at k = 2 training lifts probe accuracy from about 0.33 (untrained) to between 0.55 and 0.74 (trained), while at k = 32 a random linear projection already gets 0.948. Differences between architectures are therefore only meaningful at narrow widths, which is exactly where we found them.
- **Cost-efficiency.** For fully connected networks the symmetric model costs about twice the operations of the light-encoder / heavy-decoder model for 0.026 more probe accuracy at k = 8, and the heavy-encoder / light-decoder design costs the same as the light-encoder / heavy-decoder design while reconstructing and clustering worse.

5.4 Limitations

- **Small and easy data.** 1,797 images at 8×8, 450 test images (one accuracy point ≈ 0.2%). Findings may not transfer to MNIST, CIFAR or natural images, or to convolutional / transformer models.
- **Few seeds.** 5 seeds for the main sweep and 3 for the depth grid and the diagnosis. Paired tests with n = 5 have little power, and no multiple-comparison correction was applied; treat p-values as guides.
- **Capacity is not matched.** Variants differ by up to about 130 times in encoder or decoder parameters, so depth and parameter count are confounded (the depth grid partly separates them).
- **Shared hyper-parameters.** Only λ was tuned. A longer schedule or per-architecture tuning might shrink the collapse for the heavy-encoder model, as the 1000-epoch and higher-learning-rate runs hint.
- **One regulariser, one noise type.** The study covers a single latent regulariser and one corruption (masking).

5.5 Next steps

- Repeat on MNIST / Fashion-MNIST / CIFAR-10 with convolutional encoders and masked-patch objectives, where the masked auto-encoder premise applies.
- Test the optimisation-plateau explanation directly (learning-rate warm-up, normalisation layers, initialisation scale) and match parameter counts.
- Add invariance-inducing training (augmentation or contrastive terms) to see whether asymmetry changes robustness once the objective asks for it.

Reproducibility

Code, raw per-run results (results/*.json), summary and statistics tables, figures, tests and the interactive latent-space explorer are in the project repository (github.com/kashviagrawal04/ASYMMETRIC-ENCODER). `make experiments` regenerates every number in about 15 minutes on two CPU cores; `python build_report.py` rebuilds this document from the result files. A live, interactive latent-space explorer (the trained decoders running in the browser) is available at https://kashviagrawal04.github.io/ASYMMETRIC-ENCODER/demo/.

References

[1] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504-507, 2006.

[2] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in Proc. ICML, 2008, pp. 1096-1103.

[3] K. He, X. Chen, S. Xie, Y. Li, P. Dollar and R. Girshick, "Masked autoencoders are scalable vision learners," in Proc. CVPR, 2022, pp. 16000-16009.

[4] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533-536, 1986.

[5] A. Bardes, J. Ponce and Y. LeCun, "VICReg: Variance-invariance-covariance regularization for self-supervised learning," in Proc. ICLR, 2022.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.

[7] O. Roy and M. Vetterli, "The effective rank: A measure of effective dimensionality," in Proc. EUSIPCO, 2007, pp. 606-610.

[8] J. Venna and S. Kaski, "Neighborhood preservation in nonlinear projection methods: An experimental study," in Proc. ICANN, 2001, pp. 485-491.

[9] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," arXiv:1610.01644, 2016.

[10] L. Jing, P. Vincent, Y. LeCun and Y. Tian, "Understanding dimensional collapse in contrastive self-supervised learning," in Proc. ICLR, 2022.

[11] K. He, X. Zhang, S. Ren and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in Proc. ICCV, 2015, pp. 1026-1034.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[13] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[14] E. Alpaydin and C. Kaynak, "Optical Recognition of Handwritten Digits," UCI Machine Learning Repository, 1998.